CASE STUDY 9: STUDENT MARKS MATRIX ANALYSIS

1. Problem Statement

Student Marks Matrix Analysis is a program used to store and analyze the marks of students in different subjects using a two-dimensional list (matrix).

The program accepts marks of 3 students in 3 subjects. It calculates the total marks and average marks of each student and displays the results.

This program helps students understand how to use lists, nested loops, and basic mathematical operations in Python.

2. Objectives

1. To store student marks using a two-dimensional list.
2. To calculate the total marks of each student.
3. To calculate the average marks of each student.
4. To understand nested loops in Python.
5. To display the results in a simple format.

3. Assumptions

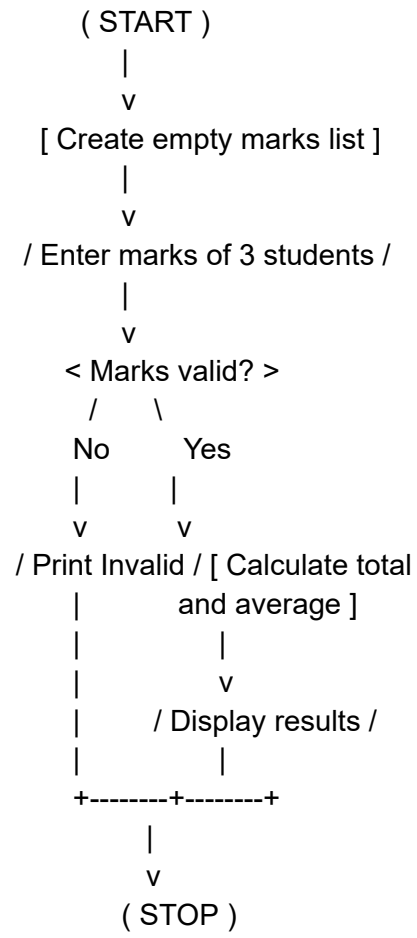
1. The program considers 3 students and 3 subjects.
2. Each subject carries a maximum of 100 marks.
3. Marks must be between 0 and 100.
4. The total is calculated by adding the marks in all 3 subjects.
5. The average is calculated by dividing the total by 3.

4. Algorithm

1. Start the program.
2. Create an empty list to store student marks.
3. Read marks for 3 students in 3 subjects.
4. Check whether each mark is between 0 and 100.
5. If any mark is invalid, display an error message and stop.
6. Store the marks of each student in the list.
7. Calculate the total marks of each student.
8. Calculate the average marks of each student.
9. Display the marks, total, and average for each student.
10. Stop the program.

5. Flowchart

Draw the following flowchart using standard symbols in your record.



6. Pseudocode

```
START
Create empty marks list

FOR each of 3 students
    INPUT marks in 3 subjects

    IF any mark is less than 0 or greater than 100
        PRINT "Invalid marks"
        STOP
    END IF

    Store marks in the list
END FOR

FOR each student
    total = sum of 3 subject marks
```

```
    average = total / 3
    PRINT total and average
END FOR

STOP
```

7. Python Source Code

```
print("STUDENT MARKS MATRIX ANALYSIS")

marks = []

for i in range(3):
    print("Enter marks for Student", i + 1)
    student_marks = []

    for j in range(3):
        mark = int(input("Enter marks for Subject " + str(j + 1) + ": "))

        if mark < 0 or mark > 100:
            print("Invalid marks")
            quit()

        student_marks.append(mark)

    marks.append(student_marks)

print("\nStudent Results")

for i in range(3):
    total = sum(marks[i])
    average = total / 3

    print("Student", i + 1)
    print("Marks:", marks[i])
    print("Total:", total)
    print("Average:", round(average, 2))
```

8. Test Cases and Expected Outputs

Test Case 1: Normal marks

Input:

Student 1: 80, 70, 90
Student 2: 60, 75, 65
Student 3: 90, 85, 95

Expected Output:

STUDENT MARKS MATRIX ANALYSIS
Enter marks for Student 1
Enter marks for Subject 1: 80
Enter marks for Subject 2: 70
Enter marks for Subject 3: 90
Enter marks for Student 2
Enter marks for Subject 1: 60
Enter marks for Subject 2: 75
Enter marks for Subject 3: 65
Enter marks for Student 3
Enter marks for Subject 1: 90
Enter marks for Subject 2: 85
Enter marks for Subject 3: 95

Student Results
Student 1
Marks: [80, 70, 90]
Total: 240
Average: 80.0
Student 2
Marks: [60, 75, 65]
Total: 200
Average: 66.67
Student 3
Marks: [90, 85, 95]
Total: 270
Average: 90.0

Explanation: The program calculates the total and average marks for all three students.

Test Case 2: All marks are zero

Input:

Student 1: 0, 0, 0
Student 2: 0, 0, 0
Student 3: 0, 0, 0

Expected Output:

Student 1
Marks: [0, 0, 0]
Total: 0
Average: 0.0
Student 2
Marks: [0, 0, 0]
Total: 0

Average: 0.0
Student 3
Marks: [0, 0, 0]
Total: 0
Average: 0.0

Explanation: When all marks are zero, the total and average for each student are also zero. The output shown above is the results section of the program.

Test Case 3: Invalid marks

Input:

Student 1, Subject 1: 120

Expected Output:

STUDENT MARKS MATRIX ANALYSIS
Enter marks for Student 1
Enter marks for Subject 1: 120
Invalid marks

Explanation: Since 120 is greater than 100, the program displays an invalid marks message and stops.

9. Output Screenshots

```
print("STUDENT MARKS MATRIX ANALYSIS")

marks = []

for i in range(3):
    print("Enter marks for Student", i + 1)
    student_marks = []

    for j in range(3):
        mark = int(input("Enter marks for Subject " + str(j + 1) + ": "))

        if mark < 0 or mark > 100:
            print("Invalid marks")
            quit()

        student_marks.append(mark)

    marks.append(student_marks)

print("\nStudent Results")

for i in range(3):
    total = sum(marks[i])
    average = total / 3

    print("Student", i + 1)
    print("Marks:", marks[i])
    print("Total:", total)
    print("Average:", round(average, 2))
```

```
= RESTART: C:/Users/sirit/OneDrive/Desktop/Python-Activity-Based-Learning/case9.
py
STUDENT MARKS MATRIX ANALYSIS
Enter marks for Student 1
Enter marks for Subject 1: 80
Enter marks for Subject 2: 70
Enter marks for Subject 3: 90
Enter marks for Student 2
Enter marks for Subject 1: 60
Enter marks for Subject 2: 75
Enter marks for Subject 3: 65
Enter marks for Student 3
Enter marks for Subject 1: 90
Enter marks for Subject 2: 85
Enter marks for Subject 3: 95

Student Results
Student 1
Marks: [80, 70, 90]
Total: 240
Average: 80.0
Student 2
Marks: [60, 75, 65]
Total: 200
Average: 66.67
Student 3
Marks: [90, 85, 95]
Total: 270
Average: 90.0
>>
= RESTART: C:/Users/sirit/OneDrive/Desktop/Python-Activity-Based-Learning/case9.
py
STUDENT MARKS MATRIX ANALYSIS
Enter marks for Student 1
Enter marks for Subject 1: 0
Enter marks for Subject 2: 0
```

```
= RESTART: C:/Users/sirit/OneDrive/Desktop/Python-Activity-Based-Learning/case9.
py
STUDENT MARKS MATRIX ANALYSIS
Enter marks for Student 1
Enter marks for Subject 1: 0
Enter marks for Subject 2: 0
Enter marks for Subject 3: 0
Enter marks for Student 2
Enter marks for Subject 1: 0
Enter marks for Subject 2: 0
Enter marks for Subject 3: 0
Enter marks for Student 3
Enter marks for Subject 1: 0
Enter marks for Subject 2: 0
Enter marks for Subject 3: 0

Student Results
Student 1
Marks: [0, 0, 0]
Total: 0
Average: 0.0
Student 2
Marks: [0, 0, 0]
Total: 0
Average: 0.0
Student 3
Marks: [0, 0, 0]
Total: 0
Average: 0.0
>>
= RESTART: C:/Users/sirit/OneDrive/Desktop/Python-Activity-Based-Learning/case9.
py
STUDENT MARKS MATRIX ANALYSIS
Enter marks for Student 1
Enter marks for Subject 1: 120
Invalid marks
>>
```

10. Conclusion

The Student Marks Matrix Analysis program stores marks using a two-dimensional list and calculates the total and average marks of each student. It uses nested loops, lists, conditional statements, and mathematical operations. This program helps beginners understand how to process student data using Python.